

```

    for (let y = 0; y < HEIGHT; y++) {
        maze[[x, y]] = WALL; // Every space is a wall at first.
    }
}

function printMaze(maze, markX, markY) {
    // Displays the maze data structure in the maze argument. The
    // markX and markY arguments are coordinates of the current
    // '@' location of the algorithm as it generates the maze.
    document.write('<code>');
    for (let y = 0; y < HEIGHT; y++) {
        for (let x = 0; x < WIDTH; x++) {
            if (markX === x && markY === y) {
                // Display the '@' mark here:
                document.write(MARK);
            } else {
                // Display the wall or empty space:
                document.write(maze[[x, y]]);
            }
        }
        document.write('<br />'); // Print a newline after printing the row.
    }
    document.write('</code>');
}

function visit(x, y) {
    // "Carve out" empty spaces in the maze at x, y and then
    // recursively move to neighboring unvisited spaces. This
    // function backtracks when the mark has reached a dead end.

    maze[[x, y]] = EMPTY; // "Carve out" the space at x, y.
    printMaze(maze, x, y); // Display the maze as we generate it.
    document.write('<br /><br /><br />');

    while (true) {
        // Check which neighboring spaces adjacent to
        // the mark have not been visited already:
        let unvisitedNeighbors = [];
        if (y > 1 && !JSON.stringify(hasVisited).includes(JSON.stringify([x, y -
2])))) {
            unvisitedNeighbors.push(NORTH);
        }
        if (y < HEIGHT - 2 &&

```